ROADMAP TWO · SYSTEMS FOR INFERENCE

Inference Engineering

the machine underneath the model

A working-knowledge OS/DBMS supplement, then a no-fluff path from GPU fundamentals to production LLM serving — built on the async and OS foundations you already have.

PARTS	PHASES	STARTING POINT	DEPTH	COMPILED
2	06	Zero CUDA	Working, not CS	Aug 2026

Why this shape

You know AI conceptually — attention, KV cache, why models generate token by token. What you don't yet have is the machine those ideas run on. **Inference engineering is almost entirely about that machine**: which memory is fast, which operation is a bottleneck, and how a serving system schedules thousands of requests onto hardware that can only do so much at once. Every technique in this field — quantization, PagedAttention, continuous batching — is a specific answer to a specific hardware constraint. Learn the constraint first and the technique stops being a name to memorise.

Part A is short by design: a few OS and DBMS ideas that keep paying off no matter what you build next, kept at the same "working knowledge" depth as your original roadmap. Part B is the real content — six phases from raw GPU hardware to a production-shaped serving stack, sized specifically to a zero-CUDA, conceptual-transformers starting point.

PART A — SUPPLEMENTARY FOUNDATIONS

	TOPIC	WHY IT'S HERE
A1	Virtual memory	PagedAttention is literally OS paging applied to KV caches — this is not an analogy, it's the same idea.
A2	mmap & file I/O	Model weight files (safetensors) are memory-mapped, not read — loading a 70B model depends on this.
A3	Indexing & key-value stores	Vector DBs and semantic caches in front of an inference server both rest on these two ideas.

PART B — INFERENCE ENGINEERING

	PHASE	WHAT YOU WALK AWAY WITH
1	GPU & hardware	Why a GPU is a throughput machine, the memory hierarchy, compute-bound vs. memory-bound.
2	Inference mechanics	Prefill vs. decode, the KV cache formula, why decode is memory-bandwidth bound.
3	Core optimizations	Quantization, FlashAttention, PagedAttention, continuous batching, speculative decoding.
4	Serving frameworks	vLLM hands-on, plus a working map of Triton and TensorRT-LLM.
5	Distributed inference	Tensor and pipeline parallelism — for models too big for one GPU.
6	Production serving	Where this plugs directly into the FastAPI/asyncio work you're already doing.

HOW THIS CONNECTS TO WHAT YOU'VE ALREADY DONE

This is not a parallel track — it's your async roadmap's Phase 1 (OS, memory, scheduling) and Phase 4 (asyncio/FastAPI serving) pointed at a specific domain. The event-loop concurrency model you built for Phase 2's echo server is the same model vLLM uses to serve thousands of concurrent generations. The GIL discussion from Phase 1 is why inference servers lean on async I/O rather than threads for request handling. Nothing here is disconnected from what you already have.

A

OS and DBMS, the working-knowledge version

Three ideas, chosen because they resurface constantly in AI infrastructure work — not a general CS survey.

A1 — VIRTUAL MEMORY AND PAGING

BOOK

OSTEP, Chapters 13, 15, 18–19, 21–22

Address spaces, address translation, paging, and swapping — the natural continuation from where you left OSTEP's concurrency chapters. Skim the mechanics; the concept to keep is: physical memory is chopped into fixed-size pages, and a page table maps logical addresses to wherever those pages actually live. That's the whole idea PagedAttention borrows.

pages.cs.wisc.edu/~remzi/OSTEP/

WHY IT MATTERS HERE

vLLM's core innovation, PagedAttention, splits each request's KV cache into fixed-size blocks that don't need to sit contiguously in GPU memory — a block table maps logical positions to physical blocks, exactly like an OS page table. The paper's own framing: "*inspired by virtual memory and paging in operating systems.*" With this section, PagedAttention is an application of something you already understand, not a clever trick.

A2 — MEMORY-MAPPED FILES

READING

The `mmap` system call, conceptually

Instead of reading a file into memory, `mmap` maps it directly into a process's address space — pages load from disk lazily, only as touched. Fifteen minutes with `man mmap` and a short blog explainer is enough.

WHY IT MATTERS HERE

The `.safetensors` format Hugging Face uses for weights is designed to be memory-mapped. Loading a 70B checkpoint doesn't read 140GB into RAM up front — it maps the file and lets pages fault in as layers are used, which is why safetensors loads dramatically faster than the older pickle-based `.bin` format. A direct payoff of A1.

A3 — INDEXING AND KEY-VALUE STORES

NPTEL

Data Base Management System — Prof. Partha Pratim Das & Prof. Samiran Chattopadhyay, IIT Kharagpur

Don't take the full course. Watch only the lectures on **indexing (B-trees/B+trees)** and **the relational model** — normalization, query optimization, and transaction theory are outside what you need.

nptel.ac.in/courses/106105175

CONCEPT

Vector indexes (HNSW, IVF) and key-value caching (Redis)

HNSW and IVF are the two index structures behind every vector database (Pinecone, Weaviate, Milvus, pgvector) — a graph you hop across for HNSW, coarse-then-fine partitioning for IVF; you need the intuition, not the construction algorithms. Redis-style LRU eviction, TTLs, and cache-aside patterns reappear inside inference servers for prefix caching and KV-cache-aware routing. If your work touches RAG or that kind of routing, it rests on one of these two structures — recognise when a slow lookup is an indexing problem and know which family fits.

PHASE ONE · PART B

01

GPU and hardware fundamentals

Starting from zero CUDA. Enough to read what a kernel does and reason about speed — not enough to write production kernels yet, and that's deliberate.

THE CORE MENTAL MODEL

REFERENCE

Modal GPU Glossary

The single best on-ramp: hypertext, cross-linked, built specifically for this gap. Read **Device Hardware** (SM, CUDA core, warp scheduler, register file) and **Device Software** (thread, warp, block, grid, memory hierarchy). Skip the host-side API section for now.

modal.com/gpu-glossary

ESSAY

Making Deep Learning Go Brrrr From First Principles — Horace He

The essay that everyone in ML systems cites, for good reason. Establishes the **roofline model** — every operation is either compute-bound or memory-bandwidth-bound, and knowing which one tells you what's even worth optimizing. Its central, load-bearing claim: most deep learning operations are memory-bound, not compute-bound. That single sentence is the reason half of Phase 3 exists.

horace.io/brrr_intro.html

CPU VS. GPU, PRECISELY

THE ONE-PARAGRAPH VERSION

A CPU core is built for **latency** — branch prediction, deep pipelines, large caches, all in service of finishing one instruction stream fast. A GPU is built for **throughput** — thousands of simple cores grouped into Streaming Multiprocessors (SMs), each running groups of 32 threads (a *warp*) in lockstep. A GPU is slower per-instruction and dramatically faster in aggregate, provided you can give it enough independent work to fill all those threads. This is the exact axis your original roadmap drew between concurrency and parallelism — a GPU is what real, hardware-level parallelism looks like at massive scale.

THE MEMORY HIERARCHY THAT ACTUALLY MATTERS

```
SRAM (on-chip, per-SM) ~19 TB/s, ~200KB total HBM (off-chip, "GPU memory") ~2–3 TB/s, 40–80GB PCIe / NVLink to host much slower still
```

Notice the gap between the first two rows — roughly 10×. Every optimization in Phase 3 is, underneath, an argument about which of these two memories an operation touches and how often.

EXERCISE, NOT A BUILD

Classify five operations

Take five PyTorch operations — a large matmul, a ReLU, a layernorm, an elementwise add, and softmax — and use the roofline model to reason about which are compute-bound and which are memory-bound before checking the essay's own examples. Getting this intuition right is worth more than any amount of further CUDA reading right now.

PHASE TWO · PART B

02

The transformer inference computation

You know attention and KV cache conceptually. This phase makes that precise enough to reason about memory and cost in numbers.

PREFILL VS. DECODE — THE CENTRAL DISTINCTION

THE IDEA EVERYTHING ELSE BUILDS ON

Generating a response happens in two phases with opposite performance characteristics. **Prefill** processes the entire input prompt in one forward pass — every token attends to every other token at once, which is a large, parallel matmul. It's *compute-bound*. **Decode** generates one token at a time, each step attending back over everything generated so far — small matmuls, run repeatedly, mostly waiting on memory bandwidth to fetch the KV cache. It's *memory-bound*. Nearly every serving optimization targets one of these two phases specifically, and knowing which is the first question to ask about any technique you read next.

KV CACHE, IN NUMBERS

```
KV cache size = 2 × num_layers × num_heads × head_dim × seq_len × batch_size × bytes_per_param
```

The leading 2 is for storing both K and V. Walk this through for a real model — say a 7B model, 32 layers, 32 heads, head_dim 128, at 4K context, fp16 (2 bytes) — and the number is uncomfortably large: it scales *linearly* with both sequence length and batch size, and it's held in the same GPU memory as the model weights. This is the single fact that explains why long-context serving and high-batch serving are both hard, and it's the problem PagedAttention exists to solve.

NUMERICAL PRECISION

CONCEPT

fp32 → fp16/bf16 → int8 → fp8/int4

Fewer bits per number means less data to move — and since decode is memory-bound, less data to move means faster decode, nearly for free. This is the throughline to quantization in Phase 3: it's not really about "smaller models," it's about reducing memory traffic on a memory-bound workload.

EXERCISE

Run the KV cache formula on three real models

Pick Llama-3-8B, a 13B model, and a 70B model. Compute KV cache size at 4K and 32K context, batch size 1 and batch size 32, in fp16. Watch how fast the numbers stop being "large" and become "larger than the GPU."

03

Core inference optimizations

Five techniques, each a direct, traceable answer to a constraint named in Phase 1 or 2 — read them in that light, not as a list to memorise.

FLASHATTENTION — ANSWERS THE MEMORY HIERARCHY

PAPER

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness — Tri Dao et al., 2022

Standard attention materialises the full N×N attention matrix in slow HBM. FlashAttention **tiles** the computation so blocks of Q, K, V stay resident in fast on-chip SRAM, fusing the whole operation into a single kernel and never writing the large intermediate matrix to HBM at all. Mathematically identical output, dramatically fewer memory reads/writes — a direct exploitation of the ~10× SRAM/HBM gap from Phase 1.

arxiv.org/abs/2205.14135

QUANTIZATION — ANSWERS MEMORY-BOUND DECODE

CONCEPT

Weight-only post-training quantization: GPTQ and AWQ

Both compress weights to 4-bit while keeping activations at higher precision, differing mainly in how they choose which weights tolerate more error. You need to know these two names and that "quantize a model" almost always means one of them in practice — not their internal calibration math.

PAGEDATTENTION — ANSWERS KV CACHE FRAGMENTATION

PAPER

Efficient Memory Management for LLM Serving with PagedAttention — Kwon et al., 2023

The vLLM paper. Before this, systems pre-allocated a contiguous chunk of GPU memory sized to each request's maximum possible length, wasting most of it to internal fragmentation. PagedAttention allocates fixed-size KV blocks on demand and tracks them with a block table — **this is A1's page table, applied to the KV cache problem from Phase 2.** Memory utilisation goes from roughly 40% to over 90%, which converts directly into more concurrent requests per GPU.

arxiv.org/abs/2309.06180

CONTINUOUS BATCHING — ANSWERS IDLE GPU TIME

PAPER

Orca: A Distributed Serving System for Transformer-Based Generative Models — Yu et al., OSDI 2022

Static batching waits for every sequence in a batch to finish before starting the next batch — a short reply sits idle waiting on a long one, and GPU utilisation collapses under variable output lengths. Orca's **iteration-level scheduling** evicts and admits requests after every single decoding step instead of every full batch, keeping the batch continuously full. This is the scheduling half of what makes vLLM fast; PagedAttention is the memory half.

BLOG

How Continuous Batching Enables 23x Throughput — Anyscale

The most-cited practical walkthrough, with real benchmark numbers comparing static, dynamic, and continuous batching side by side.

anyscale.com/blog/continuous-batching-llm-inference

SPECULATIVE DECODING — ANSWERS SEQUENTIAL DECODE LATENCY

CONCEPT

Draft-and-verify decoding

A small, fast "draft" model proposes several tokens ahead; the large model verifies all of them in one parallel forward pass, cheaper than generating them sequentially. Accepted tokens are free speed; rejected ones fall back to normal decoding. Know the shape of the idea — you won't implement this early on.

THE PATTERN ACROSS ALL FIVE

None of these are arbitrary tricks. FlashAttention exploits the SRAM/HBM gap. Quantization cuts bytes moved because decode is memory-bound. PagedAttention applies OS paging to KV cache fragmentation. Continuous batching removes the GPU idle time static batching creates. Speculative decoding attacks decode's sequential nature. Every one traces to a specific fact from Phase 1 or 2 — that traceability is the point of doing the phases in order.

04

Serving frameworks, hands-on

Go deep on one framework rather than shallow on three. vLLM is the right one to actually run.

PRIMARY — GO HANDS-ON

TOOL

vLLM

The reference implementation of everything in Phase 3 — PagedAttention and continuous batching, in one system you can install and run today. Serve a small open model (a 7B-class model fits on a single consumer GPU or a modest cloud instance), hit it with concurrent requests, and watch throughput as you vary batch size and context length. This is where Phase 2's formulas stop being abstract.

`github.com/vllm-project/vllm`

KNOW THE LANDSCAPE — READING, NOT BUILDING

TOOL

NVIDIA Triton Inference Server

A general model-serving platform, not LLM-specific — it hosts any framework's models behind one API, with its own dynamic batching and support for ensembles (chaining models into a pipeline). Read the docs' architecture overview; you don't need to deploy it yet.

TOOL

TensorRT-LLM

NVIDIA's compiler-and-runtime approach: it builds a highly optimized, hardware-specific execution engine ahead of time, trading flexibility for raw speed. The right mental model: vLLM optimizes for flexibility and fast iteration, TensorRT-LLM optimizes for squeezing out the last 10–20% of performance once you know exactly what you're deploying.

PROJECT

Serve, load-test, and explain the numbers

Stand up vLLM with an open 7B model. Load-test it with increasing concurrent request counts and record throughput and latency (p50, p99) at each level. Then explain the shape of that curve using Phase 2 and Phase 3 vocabulary — where does it plateau, and is that plateau compute-bound, memory-bound, or KV-cache-capacity-bound? That explanation is the actual skill this whole roadmap is building toward.

PHASE FIVE · PART B

05

Distributed inference

For when the model doesn't fit on one GPU. Concepts only at this stage — you don't need to configure a multi-node cluster to understand what's happening in one.

TWO WAYS TO SPLIT A MODEL

CONCEPT

Tensor parallelism

Split individual weight matrices across GPUs — each GPU computes a slice of every layer, and the GPUs exchange partial results constantly. Needs very fast interconnect (NVLink) because communication happens inside every layer; this is why tensor-parallel GPUs are almost always in the same physical server.

CONCEPT

Pipeline parallelism

Split the model by layer — GPU 1 holds layers 1–10, GPU 2 holds 11–20, and so on, passing activations forward. Communicates far less often than tensor parallelism, so it tolerates slower interconnect and can stretch across multiple machines — at the cost of GPUs sitting idle waiting on the pipeline to fill ("bubble" time).

WHY THIS IS PHASE 5, NOT PHASE 1

Both techniques are pure applications of Phase 1's interconnect hierarchy (NVLink vs. PCIe vs. network) to the problem of a model too large for one GPU's HBM. Nothing new conceptually — just Phase 1 applied at a different scale. This is why it's fine to keep this phase light: the hard ideas are already behind you.

PHASE SIX · PART B

06

Production serving — where this meets your other roadmap

This phase has almost nothing new to learn. It's your existing asyncio/FastAPI knowledge, aimed at this specific problem.

THE DIRECT CONNECTIONS

APPLIED

Async request handling around a sync GPU call

A GPU forward pass is a blocking operation from Python's perspective. An inference server accepts requests over async FastAPI routes, then hands the actual model call off to vLLM's own async engine (or a thread/process pool) so the event loop stays free to accept new connections while a batch is computing — precisely the Phase 4 "don't block the event loop" lesson from your async roadmap, applied to a GPU instead of a socket.

APPLIED

Timeouts, cancellation, and streaming responses

A client that disconnects mid-generation should free its KV cache slot immediately, not run to completion for no one. That's `asyncio` cancellation from Phase 5 of your original roadmap, with real cost attached — an un-cancelled generation is GPU memory another request can't use.

METRICS SPECIFIC TO THIS DOMAIN

Time-to-first-token (dominated by prefill) and time-per-output-token (dominated by decode) are the two latency numbers that actually matter for LLM serving — generic p50/p99 latency alone hides which phase is slow. These map directly onto the prefill/decode split from Phase 2.

CAPSTONE

A small, honest inference API

A FastAPI service in front of vLLM: streaming responses, a request timeout, graceful handling of client disconnects that frees resources immediately, and TTFT/TPOT logged per request. This is the same capstone shape as your async roadmap's Phase 5 project — pointed, finally, at a real model instead of a toy job queue.

APPENDIX

07

What was cut, and why

Inference engineering borders training, HPC, and compiler engineering. Each of these is genuinely good and genuinely out of scope for this path.

Writing CUDA kernels from scratch. You need to read and reason about kernels (Phase 1), not author production ones. That's a specialization inside inference engineering, not a prerequisite for it — revisit Simon Boehm's CUDA matmul optimization writeups once you're optimizing a specific bottleneck, not before.

Distributed *training* — ZeRO, FSDP, gradient synchronization. A different problem with different bottlenecks (gradient communication, not KV cache). Phase 5's tensor/pipeline parallelism transfers, but training-specific sharding does not belong here.

Full DBMS theory — normalization, query optimization, transaction isolation levels. Real and useful for building the applications around a model, but not for the inference path itself. The NPTEL course is there if you want it later.

Compiler-level optimization — `torch.compile` internals, Triton (the OpenAI kernel language, not TGI), XLA. The layer beneath what a serving framework does for you automatically. Worth knowing exists; not worth learning to write yourself yet.

Full GPU programming courses (PMPP-style). Excellent if you eventually specialize into kernel authoring. Overkill for using and reasoning about existing serving systems, which is the actual job of an inference engineer at most companies.

QUICK LINKS

OSTEP	`pages.cs.wisc.edu/~remzi/OSTEP/`
NPTEL — DBMS (IIT Kharagpur)	`nptel.ac.in/courses/106105175`
Modal GPU Glossary	`modal.com/gpu-glossary`
Making DL Go Brrrr — Horace He	`horace.io/brrr_intro.html`
FlashAttention paper	`arxiv.org/abs/2205.14135`
PagedAttention / vLLM paper	`arxiv.org/abs/2309.06180`
vLLM	`github.com/vllm-project/vllm`
Continuous batching — Anyscale	`anyscale.com/blog/continuous-batching-llm-inference`

If you build only two things from this document: run the KV-cache formula on a real model until the memory numbers feel inevitable rather than surprising, and stand up vLLM and explain its own throughput curve back to yourself in Phase 1–3 vocabulary. Everything else here is context for those two moments.